

Universidad Don Bosco Dirección de Educación a Distancia



DISEÑO Y PROGRAMACIÓN DE SOFTWARE

MULTIPLATAFORMA DPS941

Docente: Alexander Alberto Siguenza Campos

Estudiantes:

Jonathan Alexander Alberto, AC200739

Christian Giovanni Centeno, CS241743

José Alexander Montoya, MQ252529

Gabriel Quintanilla Rodríguez, QR230082

Actividad:

“Foro de Discusión 2”

Video:

<https://youtu.be/dNHF8cfnV-0>

Repositorio:

<https://github.com/Jonixmax/MisGastosApp.git>

Índice

Implementación de Autenticación y Arquitectura de Control de Gastos en React Native.....	1
1. Investigación y Fundamentos sobre Autenticación en Firebase....	1
1.1 Análisis Comparativo de Métodos Implementados.....	1
2. Documentación Técnica de la Implementación.....	2
2.1 Configuración Global del Servicio.....	2
2.2 Lógica de Estado y Autenticación en Interfaz (LoginScreen.js)	3
3. Arquitectura del Sistema de Control de Gastos (Fase Firestore)...	4
3.1 Diseño del Esquema de Datos.....	4
3.2 Reglas de Seguridad y Consultas Dinámicas.....	4
4. Validaciones básicas y manejo de errores.....	5
4.1 Validación perimetral de entradas.....	5
4.2 Mapeo y resolución de errores.....	5
4.3 Resolución de conflictos en la integración: Firmas y Hash Criptográfico (SHA-1).....	5
5. Evidencias de Funcionamiento de la Aplicación.....	6
Conclusiones.....	7
Referencias Bibliográficas.....	7

Introducción

La adopción de la tecnología móvil ha transformado radicalmente la gestión de las finanzas personales, brindando a los usuarios herramientas para administrar sus recursos económicos de manera ágil y accesible. En respuesta a esta necesidad, el presente proyecto detalla el desarrollo de una aplicación móvil enfocada en el control de gastos, construida bajo el framework React Native e integrada con la infraestructura en la nube de Firebase.

El sistema está diseñado para ofrecer una experiencia segura y eficiente. A través de la implementación de Firebase Authentication y Firestore, la aplicación permite a los usuarios acceder mediante credenciales tradicionales o cuentas de Google. Una vez autenticados, cuentan con las herramientas necesarias para registrar sus transacciones diarias, visualizar el historial de sus movimientos y obtener un cálculo automatizado de sus gastos mensuales.

Más allá de la solución tecnológica, nuestro informe tiene como objetivo consolidar la aplicación práctica de conocimientos en el desarrollo de interfaces móviles, la integración de flujos de seguridad y el diseño de bases de datos no relacionales.

A través de este informe, se expone la investigación teórica sobre los servicios de identidad de Firebase, la documentación técnica del proceso de implementación y la arquitectura estructural que hace posible el funcionamiento de la aplicación.

Implementación de Autenticación y Arquitectura de Control de Gastos en React Native

1. Investigación y Fundamentos sobre Autenticación en Firebase

Firebase Authentication opera como una solución integral de Backend-as-a-Service (BaaS) enfocada en la gestión de identidades. Facilita la implementación de sistemas de inicio de sesión seguros sin la necesidad de desarrollar o mantener una infraestructura de validación de credenciales propia. Para esta aplicación, la infraestructura se apoya en dos pilares de autenticación:

- **Autenticación Tradicional (Stateful/Stateless adaptado):** Validación mediante credenciales de correo electrónico y contraseña. Firebase encripta las contraseñas utilizando algoritmos de hash robustos (Scrypt modificado) y maneja la persistencia de la sesión mediante JSON Web Tokens (JWT) que se refrescan automáticamente de manera imperceptible para el usuario.
- **Identidad Federada (Google OAuth 2.0):** Permite el acceso delegando la validación de identidad a los servidores de Google. Este modelo reduce la fricción en el registro, mejora la tasa de conversión de usuarios y centraliza la seguridad en un proveedor de confianza.

1.1 Análisis Comparativo de Métodos Implementados

Característica	Correo y Contraseña	Google Sign-In (Federado)
Fricción de Usuario	Alta (requiere ingreso manual y validación)	Muy baja (un solo toque)

Característica	Correo y Contraseña	Google Sign-In (Federado)
Gestión de Credenciales	Firebase almacena hash y *salt*	Delegado a Google; Firebase solo almacena el UID y token de acceso
Complejidad de Integración	Baja (uso directo del SDK de Firebase)	Media (requiere configuración de SHA-1, Client IDs e integraciones nativas)

2. Documentación Técnica de la Implementación

La implementación se divide en la inicialización de la infraestructura central y el desarrollo de los componentes visuales e interactivos en React Native, manejando asincronía y el ciclo de vida de los componentes.

2.1 Configuración Global del Servicio

Para establecer la comunicación bidireccional con el backend de Firebase, se parametrizó el archivo de configuración `firebaseConfig.js`. Este módulo actúa como el Singleton de la aplicación para los servicios de la nube. Se importan explícitamente los submódulos necesarios

para reducir el tamaño del *bundle* final, siguiendo las mejores prácticas de optimización.

```
import { initializeApp } from 'firebase/app';
import { getAuth } from 'firebase/auth';
import { getFirestore } from 'firebase/firestore';

const firebaseConfig = {
  apiKey: "[OCULTO_PARA_PRODUCCION]",
  authDomain: "gastos-personales-proto.firebaseio.com",
  projectId: "gastos-personales-proto",
  // ... resto de variables de entorno
};

const app = initializeApp(firebaseConfig);
export const auth = getAuth(app);
export const db = getFirestore(app);
```

2.2 Lógica de Estado y Autenticación en Interfaz (LoginScreen.js)

En el componente principal de acceso, la captura de datos (correo y contraseña) se gestiona mediante el hook `useState` de React, garantizando que la interfaz responda reactivamente a las entradas del usuario. La función de acceso tradicional invoca `signInWithEmailAndPassword`, capturando excepciones de red o de validación de credenciales a través de un bloque `try-catch` para proveer retroalimentación visual mediante `Alert.alert`.

El aspecto de mayor complejidad técnica recae en el método `handleGoogleLogin`, el cual implementa la librería `@react-native-google-signin/google-signin`. El flujo lógico programado ejecuta los siguientes pasos secuenciales de seguridad y validación:

1. **Validación de Entorno:** `GoogleSignin.hasPlayServices()` asegura que el dispositivo Android cuente con los servicios de Google necesarios antes de intentar invocar el modal nativo.
2. **Petición de Identidad:** Se ejecuta `GoogleSignin.signIn()`, lo cual retorna la carga útil (payload) con la información de perfil y, de manera crítica, el `idToken` temporal.
3. **Soporte Multi-Versión:** La lógica incorpora una validación de estructura de objetos para el token, evaluando tanto `userInfo.idToken` (para versiones heredadas del SDK) como `userInfo.data.idToken` (para arquitecturas modernas).
4. **Delegación de Credencial:** Se transforma el token de Google en un objeto de credencial nativo de Firebase mediante `GoogleAuthProvider.credential(idToken)`.
5. **Consolidación de Sesión:** Finalmente, `signInWithCredential(auth, googleCredential)` completa el handshake, registrando al usuario en el sistema propio y habilitando el acceso a módulos protegidos.

3. Arquitectura del Sistema de Control de Gastos (Fase Firestore)

Una vez resuelta la capa de seguridad perimetral, la lógica de negocio se fundamenta en la manipulación de datos transaccionales. Se empleará Firestore, una base de datos NoSQL orientada a documentos, que permite escalabilidad horizontal y consultas en tiempo real.

3.1 Diseño del Esquema de Datos

Dado que Firestore no utiliza tablas estructuradas rígidas, se propone un modelo de datos desnormalizado basado en colecciones de primer nivel (Top-Level Collections) enfocado en el rendimiento de lectura. El esquema del modelo de objeto para cada transacción financiera será el siguiente:

Colección: "gastos"

Documento (Autogenerado por Firestore):

```
{
  "userId": "String (Referencia al uid de Authentication)",
  "nombre_gasto": "String (Ej. Compra de supermercado)",
  "monto": "Number (Formato flotante de doble precisión)",
  "categoria": "String (Alimentación, Transporte, Servicios, Ocio)",
  "fecha_registro": "Timestamp (Objeto nativo de Firebase)",
  "moneda": "String (Por defecto USD)"
}
```

3.2 Reglas de Seguridad y Consultas Dinámicas

El modelo propuesto exige la implementación de Firebase Security Rules para evitar la escalación de privilegios. Las reglas se configurarán para validar que `request.auth.uid == resource.data.userId`, asegurando el aislamiento total de la información (Tenant Isolation).

En la interfaz (Frontend), el dashboard requerirá consultas compuestas. Se utilizarán métodos combinados de Firestore como `query(collection(db, "gastos"), where("userId", "==", user.uid), orderBy("fecha_registro", "desc"))` para traer el arreglo de transacciones. A nivel de componente en React Native, se emplearán métodos de reducción de arreglos en JavaScript (`Array.prototype.reduce`) para iterar sobre la colección obtenida y calcular de forma dinámica los acumulados por categoría y el total del gasto mensual proyectado en la pantalla de inicio.

4. Validaciones básicas y manejo de errores.

Para obtener una estabilidad en el sistema y garantizar una buena experiencia para el usuario sin errores, se implementa una capa de validación de datos y un bloque de captura de errores con la API de Firebase.

4.1 Validación perimetral de entradas

Se aplican varios métodos para validar las entradas de una manera segura. La sanitización de campos se aplican métodos para que estén seguras las cadenas de texto no estén en blanco y se borran los espacios en blanco residuales al inicio y al final de las entradas. Se valida el formato al utilizar una expresión regular (Regex) esta funcionando para confirmar que la estructura del correo que se proporciona cumpla con la sintaxis válida por el protocolo del correo. Se establece un criterio de contraseñas el cual obliga a tener una longitud mínima de 6 caracteres para el registro, esto adaptándose a las políticas de seguridad por defecto en Firebase Authentication.

4.2 Mapeo y resolución de errores

Al tener una comunicación asíncrona por medio de servicios en la nube puede resultar en varios estados de fallos por los factores de red o de credenciales. La arquitectura interviene en estos fallos por medio de bloques `try-catch`, que intercepta el código devuelto con el error por la promesa y regresando una retroalimentación visual que sea comprensible para el usuario.

- **auth/invalid-email** : Notifica al usuario que el correo electrónico ingresado porque no tiene el formato adecuado o no es válido.
- **auth/user-not-found** y **auth/wrong-password** : Para garantizar la seguridad y evitar ataques de enumeración de cuentas, se regresa una respuesta con un mensaje de "Credenciales incorrectas o inválidas" sin especificar en qué componente está el fallo.
- **auth/network-request-failed** : Este le informa al usuario que su dispositivo tuvo una pérdida, bloqueo de red o no tiene conexión a internet, evitando así los bloqueos silenciosos dentro de la interfaz.

4.3 Resolución de conflictos en la integración: Firmas y Hash Criptográfico (SHA-1)

Durante el desarrollo e integración de la identidad federada (Google OAuth 2.0), se experimentó un bloqueo durante la autenticación. Este fallo es debido a las políticas de seguridad que exige Google Play Services para poder autorizar peticiones desde las aplicaciones cliente dentro de Android. Para poder resolver este conflicto que niega el acceso, es necesario gestionar las huellas digitales de los certificados del proyecto:

Requisito de Identidad de Google: Para poder delegar la autenticación, Google necesita que el proyecto de Firebase esté registrado de manera explícita el hash del almacén de las claves (keystore) utilizado para firmar y compilar la aplicación móvil.

Impacto Operativo: Sin este registro, el proveedor de la identidad rechaza la petición del método `GoogleSignIn.signIn()`, devolviendo un error interno de desarrollo

(DEVELOPER_ERROR), porque es imposible de identificar la autenticidad del software que solicita el token de acceso.

Discrepancia de Entornos: Este error se genera porque el servidor de Google no reconoce la firma de la aplicación. Para resolverlo se necesita extraer el hash de la llave temporal de depuración (`debug.keystore`), nativa del entorno de desarrollo de Android, y aprovisionarla en la consola de Firebase siguiendo la base de la configuración de la aplicación cliente.

5. Evidencias de Funcionamiento de la Aplicación

A continuación, se presentan diferentes capturas de pantalla de la aplicación móvil desarrollada, mostrando las principales funcionalidades implementadas, incluyendo el inicio de sesión, registro de gastos, visualización de movimientos y cálculo del total mensual.

Figura 1. Pantalla de inicio de sesión de la aplicación “Mis Gastos”, donde el usuario puede autenticarse mediante correo electrónico y contraseña o utilizando una cuenta de Google a través de Firebase Authentication.

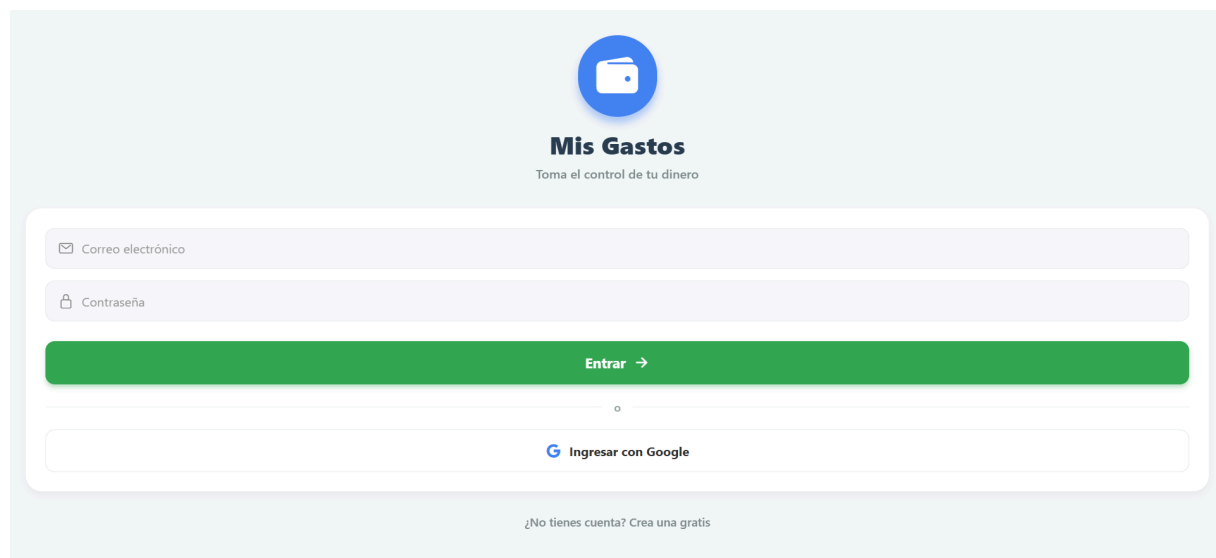


Figura 2. Proceso de autenticación mediante Google Sign-In, permitiendo al usuario seleccionar una cuenta de Google para acceder a la aplicación utilizando Firebase Authentication.

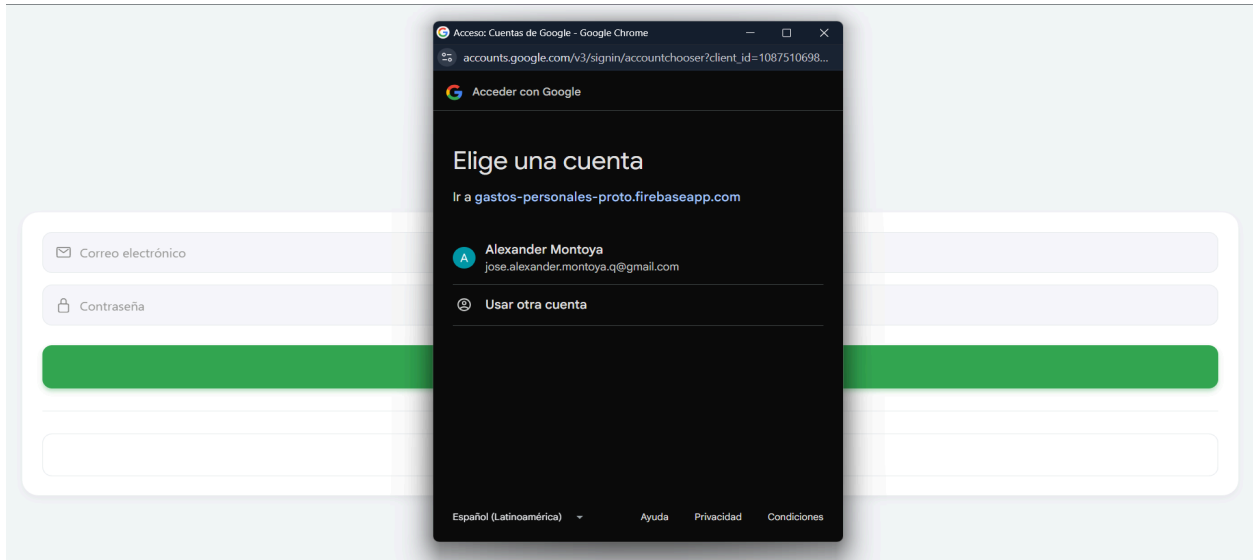


Figura 3. Validación implementada en el sistema de autenticación, mostrando un mensaje de error cuando el usuario ingresa credenciales incorrectas durante el inicio de sesión.

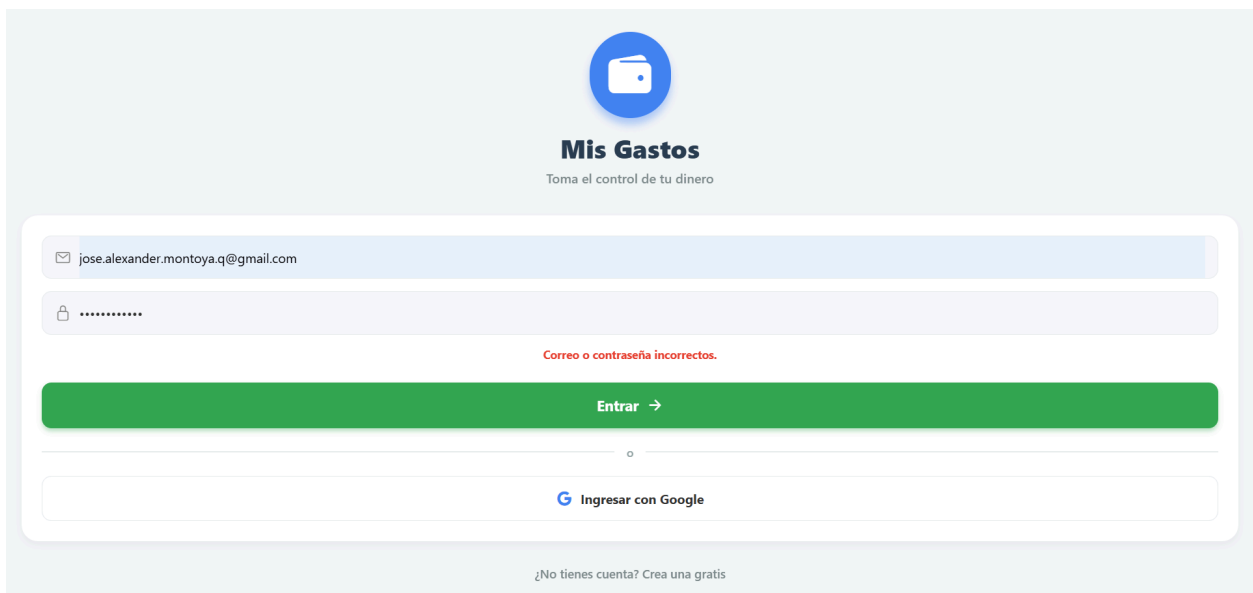


Figura 4. Pantalla principal de la aplicación móvil de control de gastos, donde se muestra el total global acumulado, las opciones para agregar nuevos gastos y los filtros por categoría y mes para la visualización de movimientos financieros.

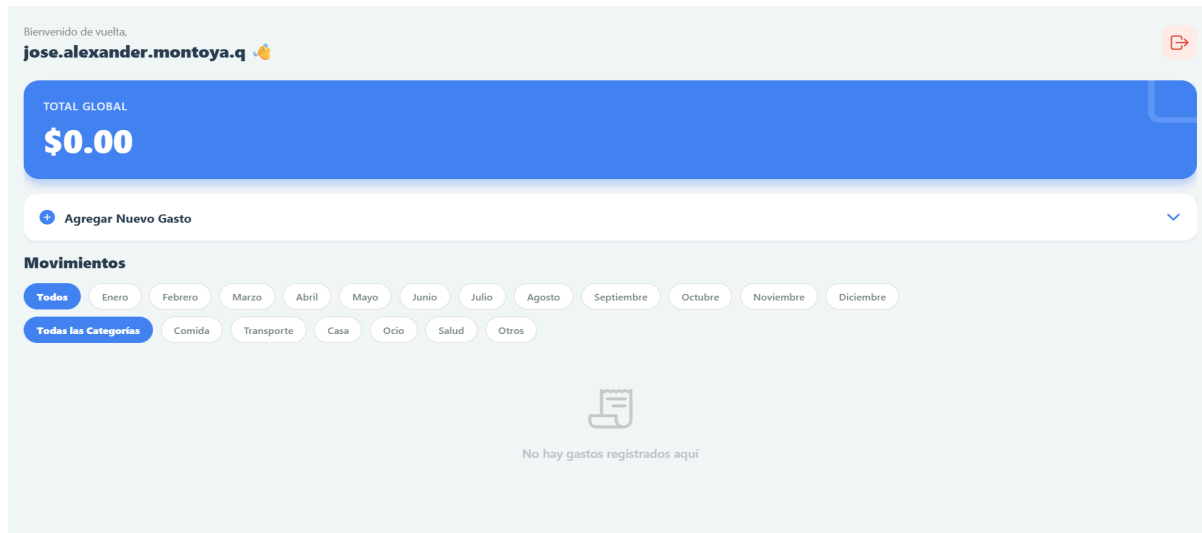


Figura 5. Pantalla de registro de usuarios implementada en React Native, permitiendo la creación de cuentas mediante correo electrónico y contraseña utilizando Firebase Authentication.

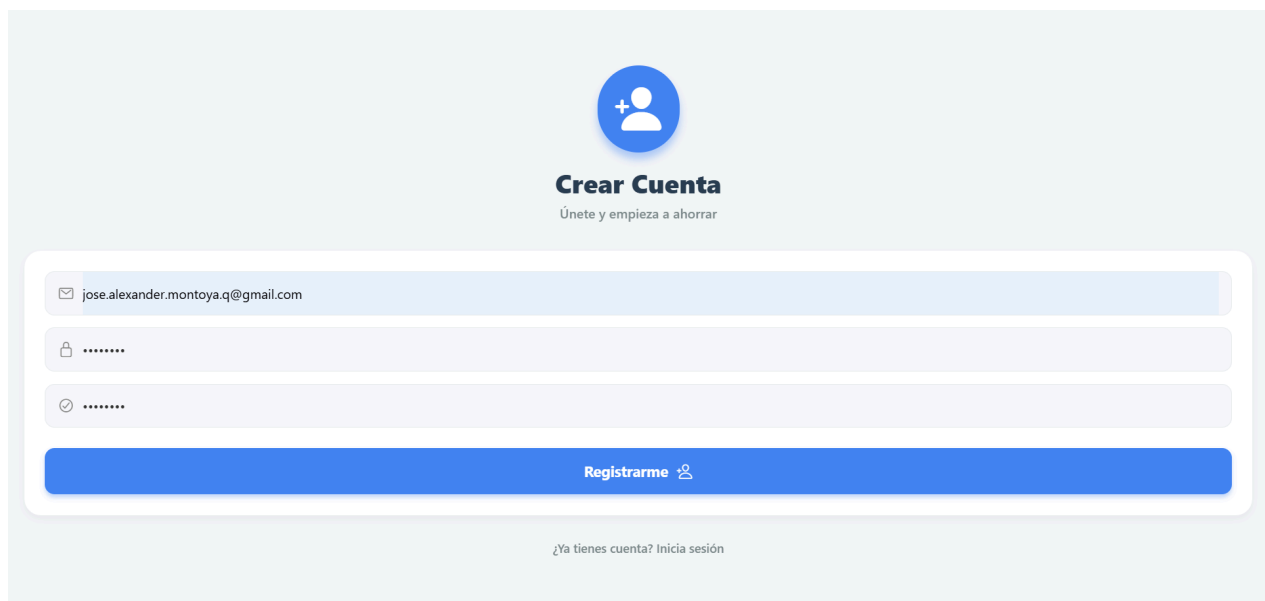


Figura 6. Interfaz de creación de gastos desarrollada en React Native, conectada a Firebase Firestore para el almacenamiento dinámico de transacciones financieras del usuario.

Bienvenido de vuelta,
jose.alexander.montoya.q 🗨️

TOTAL EN MAYO
\$0.00

+ Ocultar Formulario

Teléfono \$800

FECHA DEL GASTO
 Jueves, 7 De Mayo De 2026

CATEGORÍA
☐ Comida ☐ Transporte ☐ Casa ☒ Ocio ☐ Salud ☐ Otros

✓ Guardar Gasto

Figura 7. Visualización del total mensual calculado automáticamente a partir de los gastos registrados por el usuario, mostrando además el historial de movimientos almacenados en Firebase Firestore.

jose.alexander.montoya.q 🗨️

TOTAL EN MAYO
\$900.00

+ Agregar Nuevo Gasto

Movimientos

Todos Enero Febrero Marzo Abril **Mayo** Junio Julio Agosto Septiembre Octubre Noviembre Diciembre

Todas las Categorías Comida Transporte Casa Ocio Salud Otros

Supermercado
Comida • 07 may **\$75.00** 🗑️

Telefono
Ocio • 07 may **\$800.00** 🗑️

Internet Residencial
Ocio • 07 may **\$25.00** 🗑️

Figura 8. Historial de gastos registrados por el usuario, mostrando diferentes categorías de movimientos financieros almacenados y organizados cronológicamente dentro de la aplicación.

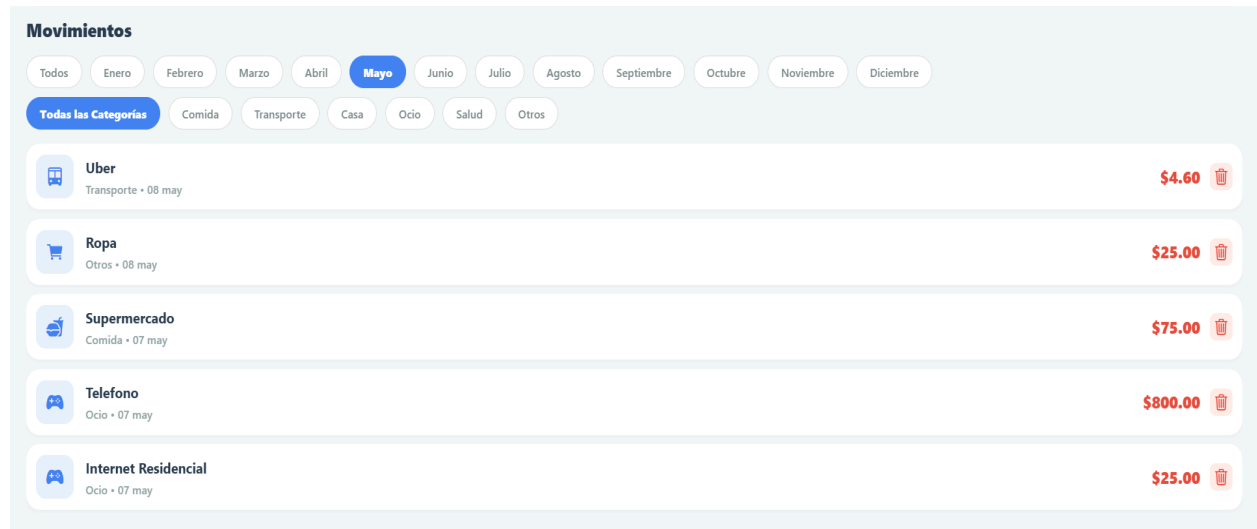


Figura 9. Confirmación de eliminación de registros implementada en la aplicación para evitar la eliminación accidental de gastos almacenados en Firebase Firestore.

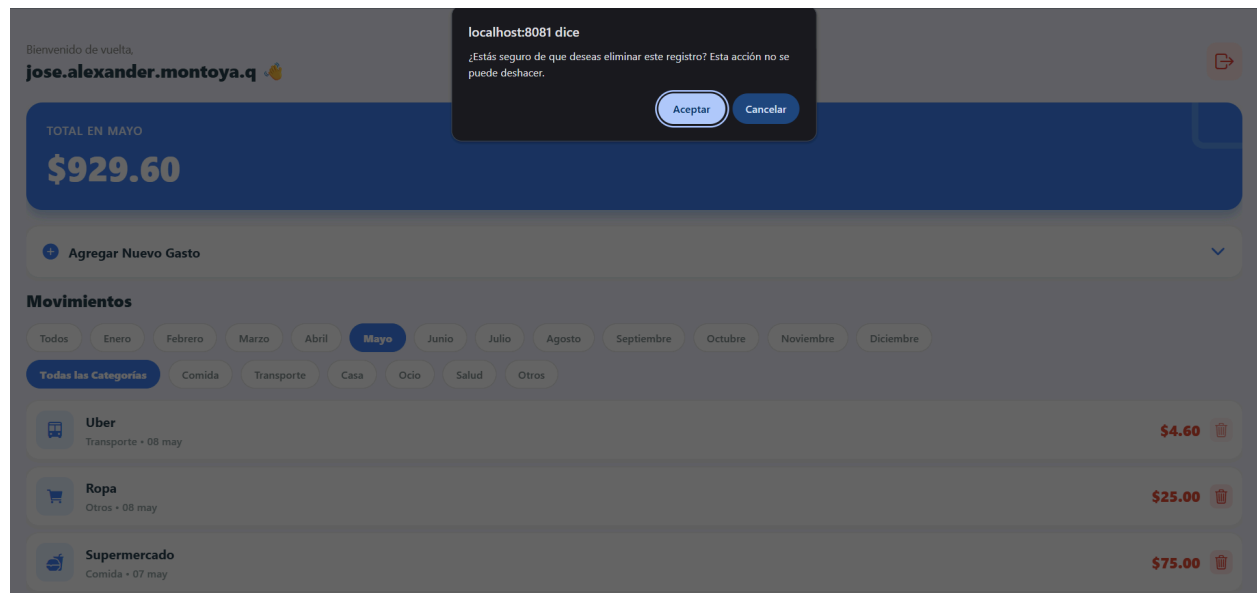


Figura 10. Estructura principal del proyecto alojada en GitHub, incluyendo los componentes, recursos y archivos utilizados durante el desarrollo de la aplicación en React Native.

Jonixmax / MisGastosApp

Escribe **/** para buscar

<> Código

Asuntos

Solicitudes de extracción

Agentes

Comportamiento

Proyectos

Wiki

Seguridad y calidad

Perspectivas

Ajustes

MisGastosApp

Público

Mirar 0

Tenedor 0

Estrella 1

maestro

2 sucursales

0 etiquetas

Ir al archivo

Agregar archivo

<> Código

Jonixmax

Fusionar la rama "master" de <https://github.com/Jonixmax/MisGastosApp>

2e9f3aa · ayer

11 Commits

.vscode	Compromiso inicial	Hace 2 semanas
aplicación	actualizar	hace 3 días
recursos/ imágenes	Compromiso inicial	Hace 2 semanas
componentes	Compromiso inicial	Hace 2 semanas
constantes	Compromiso inicial	Hace 2 semanas
manos	Compromiso inicial	Hace 2 semanas
guiones	Compromiso inicial	Hace 2 semanas
.gitignore	Compromiso inicial	Hace 2 semanas
Pantalla de gastos.js	versión mejorada	ayer
Pantalla de inicio de sesión.js	versión mejorada	ayer
LÉAME.md	Revisar el archivo README del proyecto MisGastosApp	hace 3 días

Acerca de

No se proporciona descripción, sitio web ni temas.

Léame

Actividad

1 estrella

0 espectadores

0 tenedores

Repositorio de informes

Comunicados

No se han publicado comunicados

[Crear una nueva versión](#)

Paquetes

No hay paquetes publicados.

[Publica tu primer paquete.](#)

Colaboradores

1

Conclusiones

El desarrollo e implementación de esta aplicación móvil permitió validar la eficiencia de integrar React Native con la infraestructura en la nube de Firebase. A nivel de seguridad, la arquitectura de Firebase Authentication demostró ser una solución robusta al facilitar la construcción de un sistema de acceso dual: validación tradicional por correo electrónico y delegación de identidad federada mediante Google Sign-In. La correcta implementación de estos mecanismos garantizó el aislamiento de la información (Tenant Isolation), asegurando que los registros financieros permanezcan estrictamente confidenciales y accesibles únicamente para el propietario de la cuenta.

En cuanto a la capa de persistencia de datos, la adopción de Firebase Firestore evidenció las grandes ventajas de las bases de datos NoSQL en entornos móviles modernos. Su estructura no relacional basada en documentos permitió gestionar el almacenamiento de las transacciones de manera dinámica y en tiempo real. Esta decisión arquitectónica resultó clave para optimizar la velocidad de acceso y ofrecer la escalabilidad necesaria para procesar y calcular los historiales de gastos mensuales de forma fluida.

Finalmente, desde la perspectiva del desarrollo de software multiplataforma, la ejecución de este proyecto consolidó la aplicación de patrones fundamentales en el frontend. Se logró una integración exitosa combinando el manejo avanzado de estados reactivos, la interceptación de procesos asíncronos y la implementación de validaciones estrictas en la interfaz. El resultado final es una herramienta orientada al usuario que no solo es altamente funcional, sino que también cumple con los estándares actuales de seguridad y diseño de interfaces.

Referencias Bibliográficas

Firebase. (s.f.). *Cloud Firestore documentation*. Google Developers.

<https://firebase.google.com/docs/firestore>

Firebase. (s.f.). *Firebase Authentication documentation*. Google Developers.

<https://firebase.google.com/docs/auth>

Firebase. (s.f.). *Firebase Security Rules*. Google Developers.

<https://firebase.google.com/docs/rules>

Google Developers. (s.f.). *Authenticate using Google Sign-In with Firebase*.

<https://firebase.google.com/docs/auth/web/google-signin>

MDN Web Docs. (s.f.). *Array.prototype.reduce()*. Mozilla Developer Network.

https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Array/reduce

React Native. (s.f.). *React Native documentation*. Meta Platforms, Inc.

<https://reactnative.dev/docs/getting-started>